

Detección de Anomalías Utilizando Autocodificadores

Autor: Matías Cisnero

Tutor: Dra. M. Juliana Gambini

Diciembre 2025

Universidad Nacional de Hurlingham



Índice General

1	Introducción	11
1.1	Motivación	11
1.2	Estado del Arte	11
1.2.1	Autoencoders for Anomaly Detection Are Unreliable	12
1.2.2	A comprehensive study of auto-encoders for anomaly detection	12
1.2.3	Enhancing Anomaly Detection Through Latent Space Manipulation	12
1.2.4	Ensemble of Autoencoders for Anomaly Detection in Biomedical Data	12
2	Materiales y Métodos	13
2.1	Conjunto de datos y análisis exploratorio	13
2.1.1	Análisis exploratorio de datos y preprocesamiento	14
2.2	Descripción de los métodos	15
2.2.1	Métricas de evaluación del modelo	17
3	Resultados	21
4	Conclusiones	23

Índice de Figuras

2.1	Matriz de correlación entre los atributos del conjunto de datos.	15
2.2	Arquitectura de un autocodificador: la entrada X es procesada por el codificador hasta obtener la representación latente Z , a partir de la cual el decodificador reconstruye la salida Y	16
2.3	Comparación entre las funciones de activación ReLU y GELU. Se observa que GELU presenta una transición suave alrededor de cero, lo que evita discontinuidades y favorece un entrenamiento más estable en modelos neuronales.	17
3.1		21
3.2	Matriz de confusión en valores absolutos.	22
3.3	Matriz de confusión en valores porcentuales.	22

Índice de Tablas

2.1	Descripción de las Diez Características Base del <i>Dataset</i>	13
2.2	Distribución de la variable objetivo <i>diagnosis</i>	14
2.3	Conjuntos de datos creados para evaluar la resistencia a anomalías	15
2.4	Matriz de confusión. La diagonal verde representa predicciones correctas, mientras que las celdas rojas corresponden a errores de clasificación.	18
3.1	Métricas de rendimiento del modelo evaluado en diferentes proporciones de datos anómalos en el entrenamiento	21

Resumen

El presente trabajo (o informe) aborda el problema de la detección de anomalías en conjuntos de datos de salud, enfocándose en el diagnóstico temprano y la prevención de enfermedades como la diabetes. Se emplea la arquitectura de Autocodificadores (*Autoencoders*), una clase de red neuronal no supervisada, para aprender una representación compacta y eficiente de los patrones de datos normales. El modelo se entrena utilizando el conjunto de datos **Diabetes Health Indicators Dataset**, donde se considera anómalo cualquier patrón que se desvíe significativamente de la reconstrucción esperada. La implementación se realiza mediante la biblioteca PyTorch. Los resultados de esta investigación muestran ***. Se presenta un análisis exploratorio de los datos inicial, incluyendo la matriz de correlación y ***.

Chapter 1

Introducción

Describir el problema que se desea resolver

La detección de anomalías es un campo fundamental dentro del análisis de datos, cuyo objetivo principal es la identificación de patrones que no se ajustan a un comportamiento que coincide con el de la mayoría de los registros. En el contexto de los datos de salud, una anomalía puede representar un error de medición, o una indicación temprana de una condición médica patológica, como la diabetes. Este trabajo se centra en desarrollar un sistema robusto para la detección automática de tales desviaciones. Se busca resolver la limitación de los métodos estadísticos tradicionales, los cuales a menudo no logran capturar las complejas interrelaciones no lineales presentes en conjuntos de datos de alta dimensionalidad. El problema específico que se aborda es cómo entrenar un **autocodificador** para que aprenda exclusivamente el perfil de pacientes normales o sanos a partir de un conjunto de datos que solamente incluya pacientes sanos y otro con pacientes sanos y enfermos, y cómo utilizar su error de reconstrucción para señalar con precisión qué registros se consideran anómalos.

1.1 Motivación

Explicar porqué estudiamos este problema, para qué sirve, cuál es el impacto y en qué áreas.

El diagnóstico temprano de enfermedades como la diabetes es una tarea prioritaria en la medicina moderna. En muchos casos, los datos clínicos contienen patrones sutiles que pueden pasar desapercibidos a través de métodos tradicionales de análisis. Por este motivo, resulta fundamental el desarrollo de herramientas automáticas capaces de detectar irregularidades de manera precisa y rápida.

Las redes neuronales, y en particular los autocodificadores, ofrecen un enfoque adecuado para esta tarea. Un autocodificador es un tipo de red neuronal diseñado para aprender una representación comprimida de los datos, capturando sus características más relevantes. Esta capacidad lo convierte en un candidato natural para identificar desviaciones respecto del comportamiento considerado normal.

La incorporación de este tipo de modelos en sistemas de monitoreo podría asistir a profesionales de la salud en la detección temprana de pacientes en riesgo, incluso antes de la manifestación de síntomas clínicos evidentes. Por ello, en este proyecto se emplea un autocodificador como herramienta para analizar registros de pacientes y explorar su utilidad como detector de posibles anomalías asociadas a la diabetes.

1.2 Estado del Arte

En esta sección se realiza una descripción de algunos de los métodos más importantes existentes en la bibliografía describiendo el problema y el método utilizado por cada autor.

Se cita la bibliografía.

1.2.1 Autoencoders for Anomaly Detection Are Unreliable

Bouman y Heskes (2025) [1] analizan críticamente el uso de autocodificadores clásicos como detectores de anomalías. El trabajo sostiene que, bajo ciertas condiciones, los autocodificadores pueden reconstruir correctamente ejemplos anómalos, lo cual conduce a falsos negativos incluso en casos evidentes.

Metodología.

Resultados principales.

Limitaciones.

Relevancia.

1.2.2 A comprehensive study of auto-encoders for anomaly detection

Neloy y Turgeon (2024) [2]

1.2.3 Enhancing Anomaly Detection Through Latent Space Manipulation

Walczyna et al. (2025) [3]

1.2.4 Ensemble of Autoencoders for Anomaly Detection in Biomedical Data

<https://ieeexplore.ieee.org/abstract/document/10418130>

Chapter 2

Materiales y Métodos

Describir los métodos utilizados, si corresponde.

El método empleado en este proyecto es el **Autocodificador** (*Autoencoder*). Un autocodificador es un tipo de red neuronal artificial diseñada para aprender una representación codificada de un conjunto de datos de manera no supervisada. La arquitectura se compone de dos partes principales: el **Codificador** (*Encoder*) y el **Decodificador** (*Decoder*).

2.1 Conjunto de datos y análisis exploratorio

Se describen los datos utilizados y de donde fueron extraídos, si corresponde. En caso de que los datos sean propios, describir como fueron tomados

Para la realización de este trabajo, utilicé el conjunto de datos **Breast Cancer Wisconsin (Diagnostic) Data Set**, el cual se extrajo de la plataforma Kaggle (disponible en: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>).

El conjunto de datos contiene 569 registros de muestras citológicas y 32 atributos, de los cuales 30 son características numéricas calculadas a partir de la digitalización de una imagen de aspirado con aguja fina (FNA) de una masa mamaria, más un número de identificación y el diagnóstico.

Los 30 atributos numéricos se derivan de diez características computadas para cada núcleo celular, las cuales son:

No.	Característica	Descripción
1	Radio (<i>radius</i>)	Media de las distancias del centro a los puntos del perímetro.
2	Textura (<i>texture</i>)	Desviación estándar de los valores de la escala de grises.
3	Perímetro (<i>perimeter</i>)	Medida del contorno del núcleo.
4	Área (<i>area</i>)	Superficie cubierta por el núcleo.
5	Suavidad (<i>smoothness</i>)	Variación local en las longitudes del radio.
6	... (<i>compactness</i>)	$\text{Perímetro}^2 / \text{área} - 1.0$.
7	Concavidad (<i>concavity</i>)	Severidad de las porciones cóncavas del contorno.
8	Puntos Cóncavos (<i>concave points</i>)	Número de porciones cóncavas del contorno.
9	Simetría (<i>symmetry</i>)	Nivel de simetría del núcleo celular.
10	Dimensión Fractal (<i>fractal dimension</i>)	Aproximación a la "línea de costa" menos 1.

Tabla 2.1: Descripción de las Diez Características Base del *Dataset*

Para cada una de estas diez características base, se calculan tres medidas distintas, lo que resulta en

un total de 30 atributos finales en el conjunto de datos:

1. La **media** de la característica.
2. El **error estándar** (*Standard Error*, SE) de la característica.
3. El '**peor**' valor o más grande, el cual se calcula como la media de los tres valores más grandes de la característica.

2.1.1 Análisis exploratorio de datos y preprocesamiento

Previo al entrenamiento del modelo, realicé un **análisis exploratorio de datos (EDA)**. Este proceso incluye la limpieza de datos, el manejo de valores faltantes (si los hay), la visualización de la distribución de las variables y, fundamentalmente, el cálculo y la visualización de la matriz de correlación entre todos los atributos.

Se aplicó una **estandarización** a los atributos no binarios para asegurar que todas las entradas del modelo tienen una escala comparable.

La matriz de correlación resulta crucial porque permite identificar las **dependencias lineales** entre las variables y guía las decisiones sobre el preprocesamiento de los datos, confirmando la necesidad de un modelo capaz de capturar relaciones más complejas, como el autocodificador.

La variable objetivo, *diagnosis*, presenta la siguiente distribución:

Valor	Porcentaje (%)	Cantidad de registros
B (<i>Benigno</i>)	62.74	357
M (<i>Maligno</i>)	37.26	212

Tabla 2.2: Distribución de la variable objetivo *diagnosis*

Explicar por qué estandaricé...

Aplicué una estandarización a todos los atributos. La estandarización transforma las variables para que tengan una media (μ) de cero y una desviación estándar (σ) de uno, asegurando que todas las entradas al modelo tienen una escala comparable. Este proceso se define formalmente en la ecuación 2.1.

$$z = \frac{x - \mu}{\sigma} \quad (2.1)$$

donde x es el valor original del atributo, μ es su media y σ es su desviación estándar.

Calculo la matriz de correlación (ρ), la cual mide el grado de asociación lineal entre cada par de atributos del conjunto de datos. Su definición matemática se muestra en la ecuación 2.2.

$$\rho(X, Y) = \frac{Cov(X, Y)}{s_x s_y} \quad (2.2)$$

Donde $Cov(X, Y)$ representa la covarianza entre X e Y , y s_x , s_y son sus desviaciones estándar. Finalmente, la covarianza utilizada en ese cálculo se expresa en la ecuación 2.3.

$$Cov(X, Y) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) \quad (2.3)$$

La matriz de correlación correspondiente a los principales indicadores de salud del conjunto de datos se presenta en la figura 2.1, donde pueden observarse las relaciones lineales entre los distintos atributos.

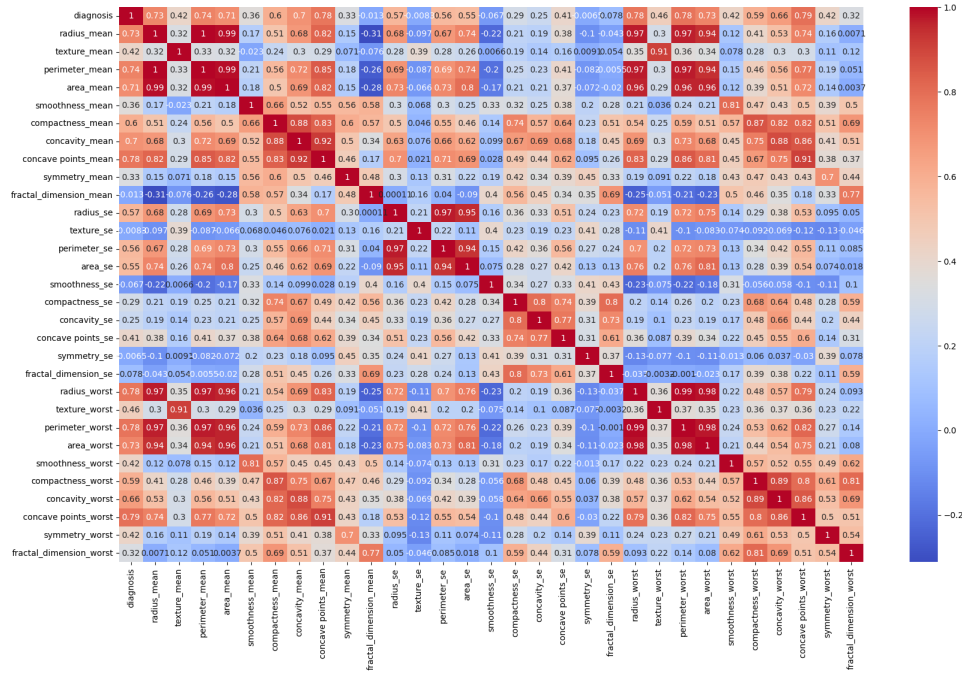


Figura 2.1: Matriz de correlación entre los atributos del conjunto de datos.

Para realizar el entrenamiento del autocodificador usé 4 conjuntos, los cuales poseen distintas proporciones de datos atípicos, esto lo realizo con el fin de evaluar la robustez del modelo ante estos. Cada conjunto de entrenamiento mantuvo un **total de 170 registros**.

ID	Proporción Anómalos (% Total)	Proporción Normales (% Total)	Registros Anómalos (Positivos)	Registros Normales (Negativos)
A	0%	100%	0	170
B	10%	90%	17	153
C	25%	75%	42	128
D	50%	50%	85	85

Tabla 2.3: Conjuntos de datos creados para evaluar la resistencia a anomalías

Estos conjuntos representan la proporción de contaminación introducida en el dataset de entrenamiento para simular escenarios donde la pureza de los datos normales no está garantizada.

Para cada conjunto realicé una división en entrenamiento y prueba. El conjunto de entrenamiento se obtuvo tomando el 80% de las muestras y manteniendo la proporción de anomalías correspondiente a su conjunto del conjunto original, mientras que el 20% restante se utilizó como conjunto de prueba, el cual es el mismo para los conjuntos A, B, C y D. De esta manera, tanto la distribución de clases como las diferentes proporciones de contaminación se mantienen consistentes en ambos subconjuntos, permitiendo evaluar el desempeño del modelo bajo escenarios comparables.

2.2 Descripción de los métodos

El método empleado en este proyecto es un autocodificador, un tipo de perceptrón multicapa caracterizado por su estructura simétrica respecto a la capa latente y por tener la misma dimensión en la entrada (d) y en la salida. Aunque suele decirse que el aprendizaje en este modelo es no supervisado, en realidad sí es supervisado, ya que la salida esperada (Y) es la propia entrada (X).

En esta arquitectura, el **codificador** comprime la entrada en una representación ubicada en la **capa latente**, que es la capa oculta central y cuya dimensión (k) suele ser menor que la de la entrada ($k < d$). Esta capa contiene la representación comprimida de los datos. Luego, el **decodificador**, que constituye la segunda mitad de la red, reconstruye la salida a partir de esa representación latente.

La arquitectura general de un autocodificador puede representarse de manera esquemática como se muestra en la figura 2.2. En ella se observa las distintas capas del modelo, desde la entrada hasta la salida.

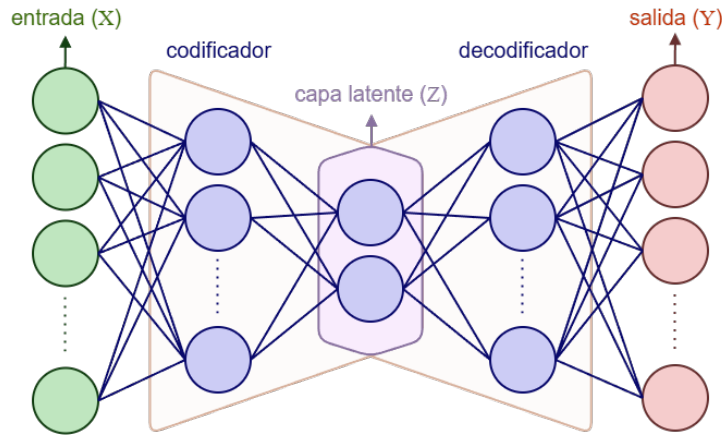


Figura 2.2: Arquitectura de un autocodificador: la entrada X es procesada por el codificador hasta obtener la representación latente Z , a partir de la cual el decodificador reconstruye la salida Y .

Como función de activación en mi autocodificador utilizo GELU debido a sus ventajas frente a la comúnmente empleada ReLU. Entre ellas se destacan: una superficie de pérdida más suave que facilita un entrenamiento más estable y rápido, la ausencia del problema de "neuronas muertas" propio de ReLU, y una mayor capacidad de modelar relaciones no lineales gracias a su naturaleza probabilística basada en la distribución normal.

La función $\text{GELU}(x)$ está definida por:

$$\text{GELU}(x) = 0.5 x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right) \quad (2.4)$$

La función $\text{ReLU}(x)$ se define como:

$$\text{ReLU}(x) = \max(0, x) \quad (2.5)$$

Como se muestra en la Figura 2.3, al graficar ambas funciones se puede apreciar la suavidad de GELU al aproximarse a 0, en contraste con el comportamiento abrupto de ReLU.

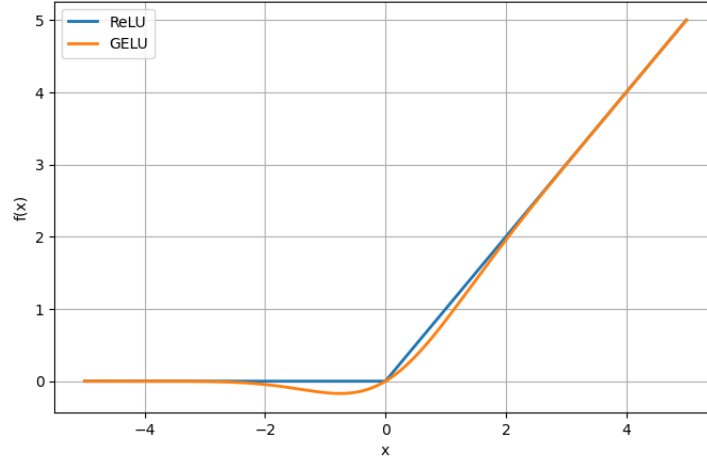


Figura 2.3: Comparación entre las funciones de activación ReLU y GELU. Se observa que GELU presenta una transición suave alrededor de cero, lo que evita discontinuidades y favorece un entrenamiento más estable en modelos neuronales.

A partir de esta arquitectura, el autocodificador puede emplearse como un método de **detección de anomalías**. Para ello, el modelo se entrena exclusivamente con la clase que representa el comportamiento normal, es decir, pacientes sin diabetes. Debido a que aprende a reconstruir adecuadamente este tipo de patrones, cuando recibe un dato t perteneciente a la misma distribución, se espera que el error de reconstrucción $\|t' - t\|$ sea pequeño.

En contraste, si se presenta un dato atípico t^* , correspondiente a un paciente con diabetes el modelo no logra reconstruirlo con la misma fidelidad, generando así un error significativamente mayor. Este comportamiento permite establecer un criterio simple y efectivo para la detección de anomalías: un dato se clasifica como anómalo si su error de reconstrucción supera un umbral ϵ predefinido. Formalmente:

$$\|t^* - t'^*\| > \epsilon \quad (2.6)$$

donde t'^* es la reconstrucción producida por la red a partir del dato t^* . Si la desigualdad se cumple, el dato se considera *anómalo*; de lo contrario, se clasifica como *normal*.

El valor del umbral ϵ se determinó de manera empírica a partir del conjunto de datos normales utilizado durante el entrenamiento. En particular, se calculó como la **media del error de reconstrucción**:

$$\epsilon = \text{media}(\|t'_i - t_i\|) \quad (2.7)$$

donde t_i corresponde a cada muestra normal y t'_i a su respectiva reconstrucción generada por el modelo. Este valor promedio proporciona un límite representativo del comportamiento esperado en datos no anómalos, permitiendo distinguir de forma efectiva aquellos casos cuyo error excede significativamente lo aprendido por la red.

2.2.1 Métricas de evaluación del modelo

Para evaluar el rendimiento del autocodificador en la tarea de detección de anomalías, se recurre a la **matriz de confusión**, la cual resume la relación entre las predicciones del modelo y las etiquetas reales. En este contexto, cada instancia se clasifica como *normal* o *anómala* según su error de reconstrucción $\|t' - t\|$ comparado con un umbral ϵ .

A partir de este criterio, se definen los siguientes casos:

- **Verdaderos Positivos (TP)**: Casos *anómalos* (personas con diabetes) cuyo error de reconstrucción satisface $\|t' - t\| > \epsilon$.

- **Falsos Negativos (FN):** Casos *anómalos* (con diabetes) pero cuyo error cumple $||t' - t|| \leq \epsilon$, es decir, el modelo no detectó la anomalía.
- **Verdaderos Negativos (TN):** Casos *normales* (sin diabetes) cuyo error satisface $||t' - t|| \leq \epsilon$.
- **Falsos Positivos (FP):** Casos *normales* (sin diabetes) cuyo error satisface $||t' - t|| > \epsilon$, siendo incorrectamente clasificados como anómalos.

Luego, con estos casos, se construye la siguiente matriz de confusión:

	Predicción	
	Anómalo	Normal
Real: Anómalo	Verdadero Positivo (TP)	Falso Negativo (FN)
Real: Normal	Falso Positivo (FP)	Verdadero Negativo (TN)

Tabla 2.4: Matriz de confusión. La diagonal verde representa predicciones correctas, mientras que las celdas rojas corresponden a errores de clasificación.

Luego de definir los cuatro posibles resultados derivados del criterio basado en el umbral ϵ , es posible cuantificar el rendimiento del modelo mediante diversas métricas. Cada una de ellas resume un aspecto distinto de la calidad de las predicciones y se deriva directamente de los valores presentes en la matriz de confusión. A continuación, se presentan las métricas utilizadas en este trabajo, junto con sus expresiones formales.

La exactitud (accuracy) cuantifica la proporción total de predicciones correctas realizadas por el modelo según la ecuación 2.8.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}} \quad (2.8)$$

La precisión (precision) indica qué proporción de las predicciones positivas corresponde realmente a casos positivos, tal como se define en la ecuación 2.9.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (2.9)$$

La sensibilidad (recall) mide la capacidad del modelo para identificar correctamente los casos positivos presentes en el conjunto de datos, de acuerdo con la ecuación 2.10.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (2.10)$$

La especificidad (specificity) evalúa la proporción de casos negativos correctamente clasificados, definida en la ecuación 2.11.

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}} \quad (2.11)$$

La exactitud balanceada (balanced accuracy), mostrada en la ecuación 2.12, combina sensibilidad y especificidad para obtener una medida equilibrada del rendimiento.

$$\text{Balanced Accuracy} = \frac{\text{Recall} + \text{Specificity}}{2} \quad (2.12)$$

El F1-Score combina las medidas de precisión y sensibilidad para ofrecer una métrica más general de la calidad del modelo, tal como se expresa en la ecuación 2.13.

$$\text{F1_Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \text{ TP}}{2 \text{ TP} + \text{FP} + \text{FN}} \quad (2.13)$$

En mi caso, resulta especialmente importante maximizar las métricas de **Recall** y **Precision**. Un alto *Recall* garantiza que la mayoría de los casos anómalos sean identificados, minimizando los falsos negativos, mientras que una alta *Precision* asegura que las instancias marcadas como anómalas realmente lo sean, reduciendo los falsos positivos. Dado que el **F1_Score** combina ambas métricas en una sola medida equilibrada del rendimiento del modelo, maximizar esta métrica resulta especialmente relevante para esta tarea.

Chapter 3

Resultados

Mostrar los resultados obtenidos utilizando gráficos, tablas, figuras, etc

Para entrenar el autocodificador se utilizó la siguiente configuración:

Insertar gráfico: <Autoencoder: In 30 → 128 → 64 → 32 → 16 → 8 → 4 → [2] → 4 → 8 → 16 → 32 → 64 → 128 → 30 Out>

Graficando los histogramas de los errores de reconstrucción:

Métrica	Conjunto 0 (0% Anómalos)	Conjunto 1 (10% Anómalos)	Conjunto 2 (25% Anómalos)	Conjunto 3 (50% Anómalos)
<i>Accuracy</i>	0.7193	—	—	—
<i>Precision</i>	0.5735	—	—	—
<i>Recall</i>	0.9286	—	—	—
<i>Specificity</i>	0.5972	—	—	—
<i>Balanced Accuracy</i>	0.7629	—	—	—
<i>F1_Score</i>	0.7091	—	—	—

Tabla 3.1: Métricas de rendimiento del modelo evaluado en diferentes proporciones de datos anómalos en el entrenamiento

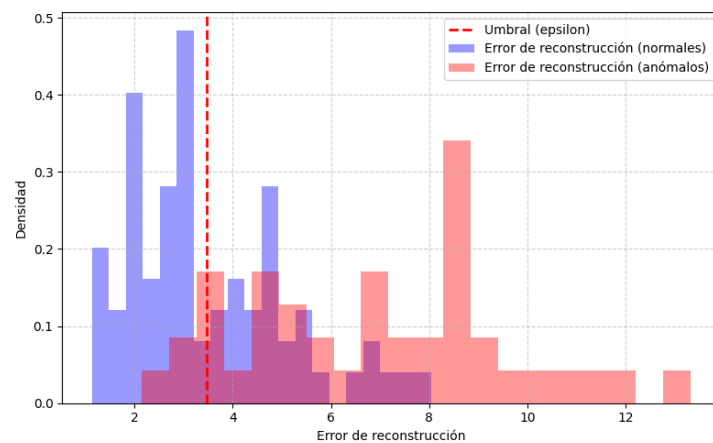


Figura 3.1

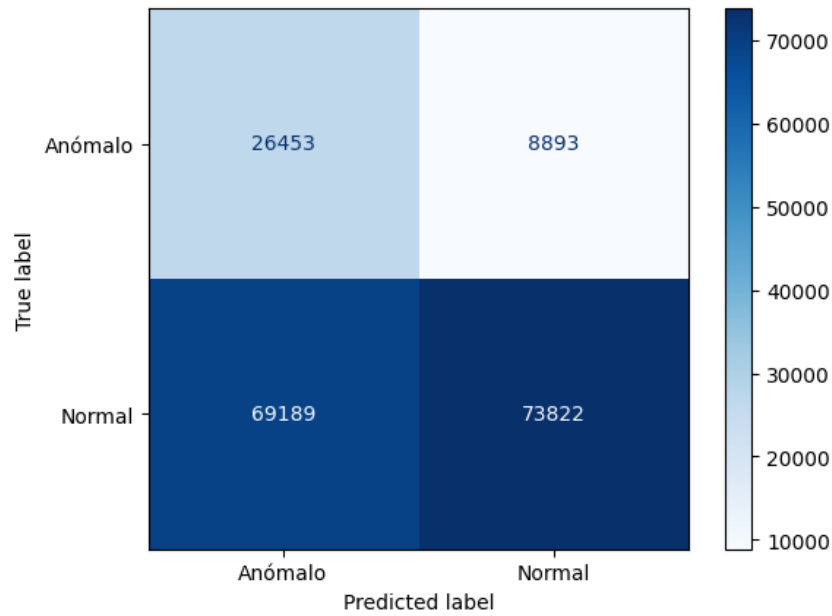


Figura 3.2: Matriz de confusión en valores absolutos.

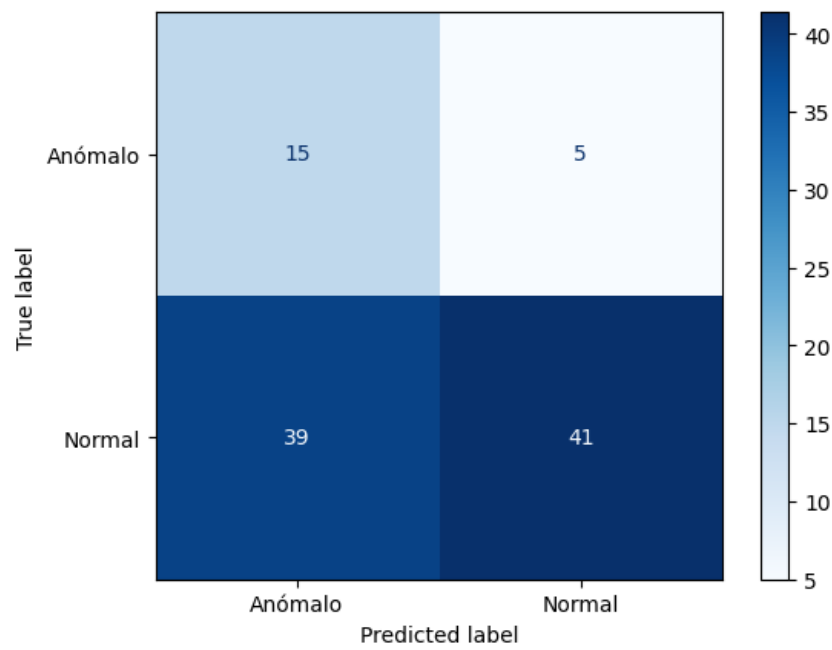


Figura 3.3: Matriz de confusión en valores porcentuales.

Chapter 4

Conclusiones

Explicar que aprendimos con la realización de este trabajo. Qué nos muestran los resultados.

Bibliography

- [1] R Bouman and T Heskes. Autoencoders for anomaly detection are unreliable. *arXiv preprint arXiv:2501.13864*, 2025.
- [2] A A Neloy and M Turgeon. A comprehensive study of auto-encoders for anomaly detection: Efficiency and trade-offs. *Machine Learning with Applications*, 17:100572, 2024.
- [3] T Walczyna et al. Enhancing anomaly detection through latent space manipulation. *Applied Sciences*, 15(1):286, 2025.